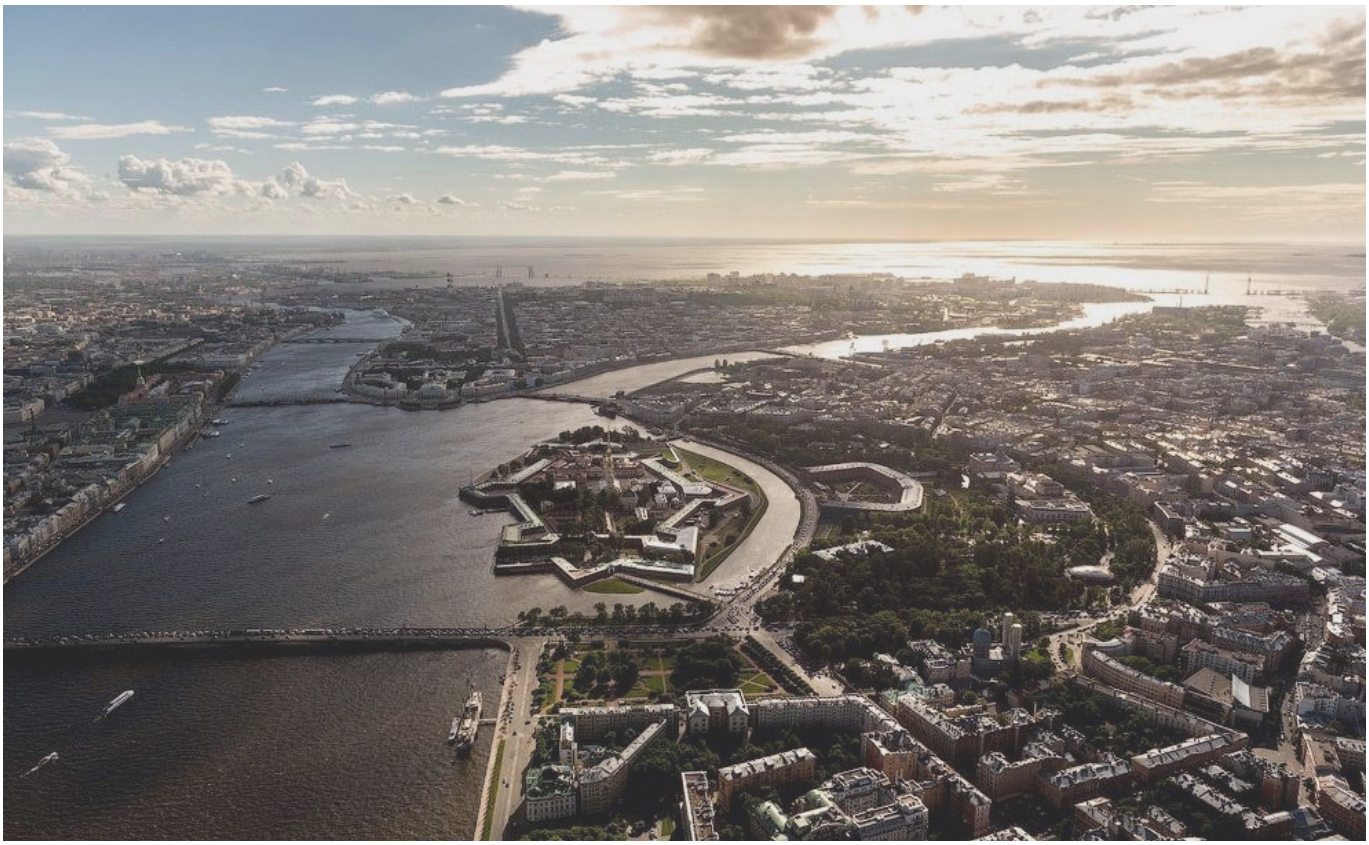
J3211, Сидельникова Дарья, 476010

ПРОЕКТИРОВАНИЕ БД

Тема 16: Поиск/покупка/аренда жилья

Лабораторная 4

(На примере https://cian.ru/)

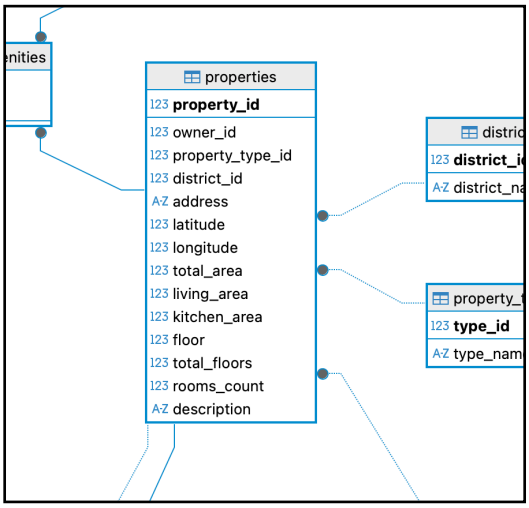


Санкт-Петербург
2025 г

ФУНКЦИИ И ТРИГГЕРЫ

Часть 1. Проблемы параллелизма

Выберите любую таблицу из вашего дз: Я выбрала properties



Типовые проблемы параллелизма

Сначала сделаю тестовую запись на которой уже потом буду смотреть изменения:

```
set search_path to realty;

insert into realty.properties (owner_id, property_type_id, district_id, address, total_area,
rooms_count)
values (1, 1, 1, 'street 1', 50, 2);
```

- 1) Грязное чтение (Dirty Read) - не получилось!

```
1 сессия:
update properties
set total_area = 200
where property_id =4;

2 сессия:
Select total_area
From  properties
Where property_id = 4;

Как было 50, так и осталось
```

- 2) Неповторяющееся чтение (Non-repeatable Read)

1 сессия:

```
select total_area from realty.properties where property_id = 4;
```

```
total_area|
----------+
     50.00|
```

2 сессия:

```
update realty.properties
set total_area = total_area + 10
where property_id = 4;
commit;
```

1 сессия повтор + commit:

```
total_area|
----------+
     60.00|
```

- 3) Фантомное чтение (Phantom Read)

1 сессия:

```
select count(*) from realty.properties where rooms_count = 2;
```

```
count|
-----+
    2|
```

2 сессия:

```
insert into realty.properties (owner_id, property_type_id,
district_id, address, total_area, rooms_count)
values (2, 1, 1, 'phantom street', 70, 2);
commit;
```

1 сессия + коммит:

```
count|
-----+
    3|
```

- 4)Аномалия сериализации (Write Skew / Lost Update)

1 сессия:

```
total_area|
----------+
     60.00|
```

```
select total_area from realty.properties where property_id = 4;

update realty.properties
set total_area = 40
where property_id = 4;
```

2 сессия:

```
select total_area from realty.properties where property_id = 4;

update realty.properties
set total_area = 35
where property_id = 4;
```

Делаем коммит в каждой:

```
total_area|
----------+
     35.00|
```

Какую из перечисленных проблем невозможно воспроизвести в postgresql при стандартных настройках? Почему?

Незакоммиченные данные не видны другим транкзакциям

1

Блокировки

Откройте 2 сессии:

- в первой сессии начните транзакцию обновляющую данные `UPDATE` или `DELETE`

```
begin;
update realty.properties
set total_area = total_area + 1
where property_id = 4;
```

- во второй сессии попытайтесь выполнить конфликтующую операцию

```
begin;
update realty.properties
set total_area = total_area + 5
where property_id = 4;
```

> Посмотрите в системную табличку pg_locks, что там можно увидеть?

```
select pid, locktype, mode, granted, relation::regclass
from pg_locks
where relation::regclass = 'realty.properties'::regclass;
```

pid	locktype	mode	granted	relation	transactionid	virtualtrar
2582	relation	RowExclusiveLock	true	properties		5/105
2582	transactionid	ShareLock	false		801	5/105
2582	transactionid	ExclusiveLock	true		802	5/105
2582	tuple	ExclusiveLock	true	properties		5/105
2583	relation	RowExclusiveLock	true	properties		6/23
2583	transactionid	ExclusiveLock	true		801	6/23

Advisory блокировки

Придумайте **2 запроса в разные таблицы**, которые **конфликтуют логически**(например, перевод денег: сначала списываем с аккаунта a, потом начисляем на аккаунт b)

Синхронизируйте эти запросы с помощью `pg_advisory_lock` [дока](https://www.postgresql.org/docs/current/explicit-locking.html#ADVISORY-LOCKS)

```
1:
select pg_advisory_lock(4);

update realty.properties
set total_area = total_area + 1
```

```
where property_id = 4;
2:
select pg_advisory_lock(4);
```

Часть 2. Хранимые процедуры и функции

Создание хп и функций

Напишите как минимум 1 **хранимую процедуру** и 1 **функцию**

```
create or replace function realty.get_property_area(p_id int)
returns numeric
language plpgsql
as $$
declare
    area numeric;
begin
    select total_area into area
    from realty.properties
    where property_id = p_id;
    return area;
end;
$$;

create or replace procedure realty.increase_area(p_id int, p_inc
numeric)
language plpgsql
as $$
begin
    update realty.properties
    set total_area = total_area + p_inc
    where property_id = p_id;
end;
                        $$;
```

```
get_property_area|
-----------------+
            43.00|
```

Циклы в процедурах

1. На базе написанных ранее **хп или функции**
   1. Напишите **хп или функцию с циклом**

2. Напишите **аналогичную хп или функцию без цикла**, используя **множественные операции** (например, один `UPDATE` с `JOIN`)
3. Сравните производительность и объясните, в каких случаях предпочтительнее каждый подход

```
create or replace procedure realty.increase_area_loop(p_id int,
p_times int)
language plpgsql
as $$
declare
    i int;
begin
    for i in 1..p_times loop
        update realty.properties
        set total_area = total_area + 1
        where property_id = p_id;
    end loop;
end;
$$;

create or replace procedure realty.increase_area_fast(p_id int,
p_times int)
language plpgsql
as $$
begin
    update realty.properties
    set total_area = total_area + p_times
    where property_id = p_id;
end;
$$;
```

```
routine_name      |routine_type|routine_schema|
------------------+------------+--------------+
increase_area_loop|PROCEDURE   |realty        |
```

Часть 3. Триггеры

Создание и тестирование триггера

1. Создайте **BEFORE или AFTER триггер**, выполняющий логирование, проверку бизнес-логики или обновление связанных данных

2. Протестируйте триггер с помощью `INSERT`/`UPDATE` и убедитесь, что он работает корректно

```
create table realty.properties_log (
    log_id serial primary key,
    property_id int,
    old_area numeric,
    new_area numeric,
    changed_at timestamp default now()
);

create or replace function realty.log_area_update()
returns trigger
language plpgsql
as $$
begin
    if new.total_area is distinct from old.total_area then
        insert into realty.properties_log(property_id, old_area,
new_area)
        values (old.property_id, old.total_area, new.total_area);
    end if;
    return new;
end;
$$;

create trigger trg_log_area_update
after update on realty.properties
for each row
execute function realty.log_area_update();
```

```
routine_name    |routine_schema|
---------------+--------------+
log_area_update|realty        |

oid  |tgrelid|tgparentid|tgname             |tgfoid|tgtype|tgenabled|tgisinterna
-----+-------+----------+-------------------+------+------+---------+-----------
17215|  16489|         0|trg_log_area_update| 17214|    17|O        |false

log_id|property_id|old_area|new_area|changed_at              |
------+-----------+--------+--------+------------------------+
     1|          4|  113.00|  118.00|2025-12-10 14:00:32.403|
```

Циклический триггер

1. Создайте **циклический триггер** (например: `UPDATE` в таблице A вызывает триггер → обновляет таблицу B → триггер в B обновляет A)
2. Выполните операцию, запускающую цикл, и посмотрите что произойдет

> Как PostgreSQL обрабатывает такую ситуацию?
>

```
create or replace function realty.cycle_trigger_fn()
returns trigger
language plpgsql
as $$
begin
    update realty.properties
    set rooms_count = coalesce(rooms_count, 0) + 1
    where property_id = new.property_id;

    return new;
end;
$$;

create trigger trg_cycle_properties
after update on realty.properties
for each row
execute function realty.cycle_trigger_fn();

update realty.properties
set total_area = total_area + 1
where property_id = 4;
```

```
routine_name     |
-----------------+
cycle_trigger_fn|
```

```
tgname               |
---------------------+
trg_cycle_properties|
```

```
SQL Error [54001]: ERROR: stack depth limit exceeded
  Hint: Increase the configuration parameter "max_stack_depth" (currently 2048kB), after ensuring the platform's stack depth limit is adequate.
  Where: SQL statement "SELECT 1 FROM ONLY "realty"."users" x WHERE "user_id" OPERATOR(pg_catalog.=) $1 FOR KEY SHARE OF x"
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
PL/pgSQL function cycle_trigger_fn() line 4 at SQL statement
SQL statement "UPDATE realty.properties
    SET rooms_count = COALESCE(rooms_count, 0) + 1
    WHERE property_id = NEW.property_id"
```

Часть 4. Низкоуровневое хранение данных в PostgreSQL

Подключите расширение `pageinspect` https://www.postgresql.org/docs/current/pageinspect.html

```
create extension if not exists pageinspect;
```

1. Выберите одну из своих таблиц (например, `artist`).
2. Получите «сырой» блок (например, блок 0) с помощью

```
select get_raw_page('realty.properties', 0);
```

1. Передайте его в `page_header()` и проанализируйте поля

```
select * from page_header(get_raw_page('realty.properties', 0));
```

> Что значат поля в полученной таблице?

```
lsn      |checksum|flags|lower|upper|special|pagesize|version|prune_xid|
---------+--------+-----+-----+-----+-------+--------+-------+---------+
0/1C37578|       0|    5|  412| 7704|   8192|    8192|      4|0        |
```